Python Workshop

Modern Python Tooling

Rishi Vora

Part 1: The better package manager

Traditional Python Environments

Traditional Python Environments

- **Global environment**: (Not recommended)
  - All projects share the same environment.
  - Risk of version conflicts and system tool issues.

Traditional Python Environments

- **Global environment**: (Not recommended)
  - All projects share the same environment.
  - Risk of version conflicts and system tool issues.
- **Virtual environments with `venv`**:
  - Isolated environments for each project.
  - `python -m venv .venv` & `source .venv/bin/activate`.

Traditional Python Environments

- **Global environment**: (Not recommended)
  - All projects share the same environment.
  - Risk of version conflicts and system tool issues.
- **Virtual environments with `venv`**:
  - Isolated environments for each project.
  - `python -m venv .venv` & `source .venv/bin/activate`.
- **Conda**:
  - Package manager for Python and non-Python dependencies.
  - `conda create -n my-env python=3.12` & `conda activate my-env`.
  - Powerful for scientific stacks, but often slow and "heavy".

Traditional Dependency Management

Traditional Dependency Management

`requirements.txt`:

- Simple list of packages.
- Lacks metadata (like the required Python version).
- Doesn't lock transitive dependencies.

Traditional Dependency Management

`requirements.txt`:

- Simple list of packages.
- Lacks metadata (like the required Python version).
- Doesn't lock transitive dependencies.

Sample

```
# This is a comment and will be ignored by pip
pandas==2.1.0        # Exact version
lxml≥4.9.2           # Minimum version
numpy~=1.24.0        # Compatible release (≥1.24.0, <1.25.0)
requests             # Latest version
git+https://github.com/psf/requests.git@main#egg=requests # Install from a public GitHub repository
```

`environment.yml` (Conda):

- Environment definition including Python version.
- Platform-specific exports make sharing difficult.

`environment.yml` (Conda):

- Environment definition including Python version.
- Platform-specific exports make sharing difficult.

Sample

```yaml
name: my-env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy ⩾ 1.24
  - scipy
  - matplotlib
```

`pyproject.toml`:

- Modern and official standard for Python project metadata and dependencies.
- Supported by many modern tools.

`pyproject.toml`:

- Modern and official standard for Python project metadata and dependencies.
- Supported by many modern tools.

Sample

```toml
[project]
name = "my-project"
version = "0.1.0"
dependencies = [
  "numpy ⩾ 1.24",
  "pandas",
]
requires-python = " ⩾ 3.9"
```

The Problems with Traditional Workflows

The Problems with Traditional Workflows

Dependency Locking

- Transitive dependencies are not locked in `requirements.txt`.
- Leads to “works on my machine” bugs.
- Modern managers (npm, cargo) use lockfiles.

The Problems with Traditional Workflows

Dependency Locking

- Transitive dependencies are not locked in `requirements.txt`.
- Leads to "works on my machine" bugs.
- Modern managers (npm, cargo) use lockfiles.

Example

`scikit-learn==1.3.0` directly depends on `numpy`, `scipy`, and `joblib`, but `scipy` itself depends on `numpy`, and `numpy` depends on several C libraries. If you specify `scikit-learn==1.3.0` in `requirements.txt`, pip will install whatever versions of `scipy`, `joblib`, and their dependencies it *deems compatible at install time*. This can lead to different environments having different transitive dependency versions.

Sample `cargo.lock`

```
[[package]]
name = "serde"
version = "1.0.163"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "2113ab51bed1e7f57b4aa1c2e48ed660eb3d3f7f44d4bcf8cfd3bf63ee574ad1"

[[package]]
name = "serde_json"
version = "1.0.104"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "076066c5a1078eac5cdd75fc1635cd020c207a6d3dcbf858667951a8ec57272f"
dependencies = [
"itoa",
"ryu",
"serde",
]

[[package]]
name = "itoa"
version = "1.0.9"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "af150ab688ff2122fcef229be89cb50dd66af9e01a4ff320cc137910150ad5b15"

[[package]]
name = "ryu"
version = "1.0.15"
source = "registry+https://github.com/rust-lang/crates.io-index"
checksum = "1ad4cc8da4ef723ed60bced201181d83791ad433213d8c24efffda1eec85d741"
```

Solution: Lockfiles

This shows exact versions of all direct and transitive dependencies, ensuring **reproducible environments across different machines and over time**.

The “Conda + Pip” Trap

- Mixing them leads to environment inconsistency.
- Pip might overwrite Conda-installed packages.
- Standard Conda does not provide a cross-platform lock file by default.

Installing pip packages

Copy page

Most of the popular packages from the PyPI repository are available in either Anaconda’s public repository, Anaconda.org, or conda-forge. However, you might need to use pip if a package or specific version is not available through conda channels.

Installing packages using pip modifies your conda environment, but conda isn’t aware of these modifications. As a result, when conda later attempts to modify the environment, there’s a high probability that dependency conflicts will arise between the conda-tracked packages and the untracked pip packages, which can lead to a broken environment.

Figure 1: From the Anaconda documentation

Solution

So what does a modern Python package manager look like? Let's compare the traditional tools with the ideal solution.

Solution

So what does a modern Python package manager look like? Let's compare the traditional tools with the ideal solution.

Feature	`requirements.txt`	`environment.yml`	`pyproject.toml`	`pyproject.toml` + Lockfile
Scope	Direct dependencies only	Environment specification	Project metadata (PEP 621)	Full project snapshot
Transitive Locking	No	Requires external tool	No	Yes (Native)
Deterministic	No	Requires external tool	No	Yes
Reproducibility	Low	Medium (Platform-bound)	Low	High (Universal)

uv - The Modern Solution

uv is a Python package and project manager written in Rust.

uv - The Modern Solution

uv is a Python package and project manager written in Rust.

- **Standard Metadata**: Fully embraces `pyproject.toml` (PEP 621) for project configuration.
- **Native Transitive Locking**: Automatically generates and uses a cross-platform lockfile (`uv.lock`).
- **Deterministic**: Ensures the exact same versions of all dependencies are installed everywhere.
- **Universal Reproducibility**: Works seamlessly across different operating systems without platform-bound exports.

What more does uv offer?

What more does uv offer?

- Handles project creation and setup.
- Installs and manages Python versions.
- Handles virtual environments.
- Runs and installs tools published as Python packages.
- Runs Python scripts, with support for inline dependency metadata.
- Includes a pip-compatible interface for a performance boost with a familiar CLI.
- Disk-space efficient, with a global cache for dependency deduplication.

What more does uv offer?

- Handles project creation and setup.
- Installs and manages Python versions.
- Handles virtual environments.
- Runs and installs tools published as Python packages.
- Runs Python scripts, with support for inline dependency metadata.
- Includes a pip-compatible interface for a performance boost with a familiar CLI.
- Disk-space efficient, with a global cache for dependency deduplication.
- 10-100x faster than pip.

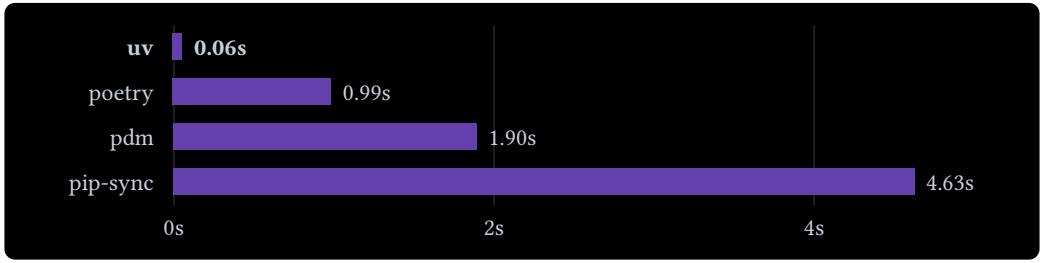

Figure 4: Installing Trio's dependencies with a warm cache

Python Management

```
# List all available Python versions
uv python list

# Install Python 3.12
uv python install 3.12
```

As simple as that! `uv` handles downloading, installing, and managing multiple Python versions seamlessly.

Project Management

```
# Create a new project in the current directory
uv init

# Pin the Python version for the project
uv python pin 3.12

# Install dependencies
uv install numpy pandas==2.3.3 matplotlib

# See the dependency tree
uv tree

# Remove a dependency
uv remove matplotlib
```

```
# Update dependencies
uv lock --upgrade-package numpy
uv lock --upgrade

# Run a Python script
uv run script.py

# Install a tool
uv tool install ruff

# Run a tool without installing
uvx ruff

# Enter the project shell
uv run python
```

Project Management

```python
# Create a new project in the current directory
uv init

# Pin the Python version for the project
uv python pin 3.12

# Install dependencies
uv install numpy pandas==2.3.3 matplotlib

# See the dependency tree
uv tree

# Remove a dependency
uv remove matplotlib
```

```python
# Update dependencies
uv lock --upgrade-package numpy
uv lock --upgrade

# Run a Python script
uv run script.py

# Install a tool
uv tool install ruff

# Run a tool without installing
uvx ruff

# Enter the project shell
uv run python
```

Notice how we don't need to activate the environment with `source .venv/bin/activate` or `conda activate my-env`. The `uv run` command automatically uses the correct environment for the project.

Want to share the project with a colleague? Deploy it on cluster? Or just ensure it works in 6 months when you forget all the details?

Want to share the project with a colleague? Deploy it on cluster? Or just ensure it works in 6 months when you forget all the details?

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12
python3.12 -m venv .venv
source .venv/bin/activate
pip-compile requirements.in -o requirements.txt
pip install -r requirements.txt
```

```
uv sync
```

Figure 6: From the guide by SaaS Pegasus

Part 2: The better notebook

Traditional Notebook Experience

Most people use **Jupyter** Notebooks (`.ipynb` files) for interactive data analysis and exploration. Jupyter provides a web-based interface where you can write and execute code in cells, visualize outputs, and document your workflow with Markdown.

Traditional Notebook Experience

Most people use **Jupyter** Notebooks (`.ipynb` files) for interactive data analysis and exploration. Jupyter provides a web-based interface where you can write and execute code in cells, visualize outputs, and document your workflow with Markdown.

Let's look at a demo that showcases all the pain points with Jupyter.

Marimo - A Jupyter Replacement

Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lbs	Acceleration	Year	Origin
toyota corona mark ii	24	4	113	95	2372	15	1970-01-01T00:00:00.000Z	Japan
plymouth duster	22	6	198	95	2833	15.5	1970-01-01T00:00:00.000Z	USA
amc hornet	18	6	199	97	2774	15.5	1970-01-01T00:00:00.000Z	USA
ford maverick	21	6	200	85	2587	16	1970-01-01T00:00:00.000Z	USA
datsun pl510	27	4	97	88	2130	14.5	1970-01-01T00:00:00.000Z	Japan
volkswagen 1131 deluxe sedan	26	4	97	46	1835	20.5	1970-01-01T00:00:00.000Z	Europe
peugeot 504	25	4	110	87	2672	17.5	1970-01-01T00:00:00.000Z	Europe
audi 100 ls	24	4	107	90	2430	14.5	1970-01-01T00:00:00.000Z	Europe
saab 99e	25	4	104	95	2375	17.5	1970-01-01T00:00:00.000Z	Europe

Figure 7: A dashboard built with Marimo

Feature	Jupyter	Marimo
State Consistency	Hidden state (cells run out of order). 36% of notebooks found non-reproducible.	Reactive (cells re-run automatically). Guarantees consistent state.
File Format	JSON (`.ipynb`). Difficult to version control.	Pure Python (`.py`). Git-friendly and human-readable.
UI Interactivity	Requires extra libraries like `ipywidgets`.	Built-in UI elements (sliders, etc.) synced with Python.
Dependencies	External environment management.	Can do inline sandboxing (package requirements serialized in the file).
Reusability	Not possible to import or run as a standalone script.	Can be imported as a module or executed from CLI.
App Deployment	Requires significant extra effort.	Every notebook is an interactive web app.

Getting Started with a Tutorial

```
# Install marimo
uv add marimo

# Open the tutorial notebook
uv run marimo tutorial for-jupyter-users
```

Getting Started with a Tutorial

```
# Install marimo
uv add marimo

# Open the tutorial notebook
uv run marimo tutorial for-jupyter-users
```

Creating and Editing Notebooks

```
# Create a new notebook
uv run marimo new

# Edit an existing notebook
uv run marimo edit notebook.py
```

Features

- Integration with **uv**.
- Source control friendliness.
- Outputs are not stored within the notebook.
- Ability to import as a module or run from CLI.
- AI integration.
- Integration with VS Code.
- Import from Jupyter. Export to Jupyter, HTML, or PDF.
- Deploy as an interactive web app or slides, run in the browser via Wasm.
- Molab as a cloud alternative to Google Colab.

Thank you!

Questions?